

Complete DSA Mastery & 60-90 Day Execution Bible

Comprehensive Pattern Breakdown, Daily Routine, Exam Strategy & Core 75 Checklist

1. Executive Strategy & Mindset Framework

DSA mastery is not about solving 1000 questions blindly. It is about **Pattern Recognition Efficiency**. When presented with an unseen problem in an interview, your brain must execute a fast decision tree to identify the underlying pattern within 60 seconds.

THE 4-STEP INTERVIEW PROTOCOL

The 4-Step Problem Solving Algorithm in Interviews:

- 1. Understand & Constraints Analysis (2 mins):** Read constraints. If $N \leq 10^5$, aim for $O(N)$ or $O(N \log N)$. If $N \leq 20$, exponential $O(2^N)$ backtracking works.
- 2. Pattern Identification (2 mins):** Match problem clues to known patterns (e.g. 'Contiguous subarray' -> Sliding Window or Prefix Sum; 'Next greater element' -> Monotonic Stack).
- 3. Dry Run & Edge Cases (3 mins):** Test with minimal input, empty input, single element, duplicates, negative numbers.
- 4. Clean Code Implementation (12 mins):** Write production-ready code with meaningful variable names, handling nulls/bounds.

2. Topic-by-Topic Deep-Dive & Code Templates

Topic 1: Arrays, Strings, Two Pointers & Sliding Window

Core Patterns & Clues:

- **Two Pointers:** Sorted array / opposite ends (#15 3Sum, #11 Container With Most Water, #42 Trapping Rain Water).
- **Sliding Window (Variable Size):** Find longest/shortest subarray satisfying condition K (#3 Longest Substring Without Repeating, #76 Minimum Window Substring).
- **Prefix Sum + HashMap:** Find count of subarrays with sum equal to K (#560 Subarray Sum K, #525 Contiguous Array). Clue: Subarray sum from L to R = $\text{Prefix}[R] - \text{Prefix}[L-1] = K \rightarrow \text{Prefix}[L-1] = \text{Prefix}[R] - K$.
- **Kadane's Algorithm:** Maximum subarray sum. Track $\text{maxEndingHere} = \max(\text{num}, \text{maxEndingHere} + \text{num})$.

```
// Sliding Window Variable Size Template
int left = 0, maxLen = 0;
Map<Character, Integer> counts = new HashMap<>();

for (int right = 0; right < s.length(); right++) {
    char c = s.charAt(right);
    counts.put(c, counts.getOrDefault(c, 0) + 1);

    while (!isValid(counts)) { // Shrink window
        char leftChar = s.charAt(left);
        counts.put(leftChar, counts.get(leftChar) - 1);
        left++;
    }
    maxLen = Math.max(maxLen, right - left + 1);
}
```

Topic 2: Stack & The Monotonic Stack Progression

Monotonic Stack is used when you need to find the *next greater* or *previous smaller* element in $O(N)$ time.

MONOTONIC STACK MASTERY ROUTE

The 6-Step Monotonic Stack Progression Path:

1. **#20 Valid Parentheses (Easy)**: Matching brackets using LIFO stack.
2. **#496 Next Greater Element I (Easy)**: Decreasing monotonic stack to store elements waiting for a greater next element.
3. **#739 Daily Temperatures (Medium)**: Store indices in monotonic stack; distance = currentIndex - stack.pop().
4. **#503 Next Greater Element II (Medium)**: Circular array simulation using modulo $i \% N$ with $2*N$ loop.
5. **#84 Largest Rectangle in Histogram (Hard)**: Maintain increasing stack of height indices. When current height is smaller, pop and compute $\text{height}[\text{popped}] * (\text{current_index} - \text{stack.peek()} - 1)$.
6. **#85 Maximal Rectangle (Hard)**: Convert 2D binary matrix into 1D histogram per row, applying #84 for each row.

Topic 3: Linked List Pointer Manipulation

- **Fast & Slow Pointers (Floyd's Cycle Detection)**: #141 Cycle, #142 Cycle II, #876 Middle of LL.
- **In-place Reversal**: Maintain prev = null, curr = head, next = null. (#206 Reverse LL, #92 Reverse LL II).
- **Complex Reorder & K-Group**: #143 Reorder List = (1) Find Middle + (2) Reverse 2nd Half + (3) Interleave. #25 Reverse Nodes in K-Group.

Topic 4: Binary Trees, BST & Tree DP

- **DFS (Preorder/Inorder/Postorder)**: Postorder is essential for bottom-up computation (#543 Diameter of Tree, #110 Balanced Tree).
- **Tree DP**: At each node, compute values from left & right subtrees and pass max upward (#124 Binary Tree Max Path Sum).
- **BST Property**: Inorder traversal of BST yields strictly sorted values (#98 Validate BST, #230 Kth Smallest).
- **BFS Level Order**: Using Queue for level-by-level processing (#102 Level Order, #199 Right Side View).

Topic 5: Graphs, Topological Sort & Shortest Path

- **Grid DFS/BFS:** #200 Number of Islands, #994 Rotting Oranges (Multi-source BFS using queue).
- **Topological Sort (Kahn's Algorithm / Indegree BFS):** For DAG ordering and cycle detection (#207 Course Schedule I, #210 Course Schedule II).
- **Disjoint Set Union (DSU / Union-Find):** Path compression + rank optimization (#684 Redundant Connection, #547 Provinces).
- **Dijkstra's Algorithm:** Shortest path in weighted graph using Priority Queue (Min-Heap) (#743 Network Delay Time).

```
// Kahn's Algorithm for Topological Sort / Course Schedule
int[] indegree = new int[numCourses];
for (int[] edge : prerequisites) indegree[edge[0]]++;

Queue<Integer> q = new LinkedList<>();
for (int i = 0; i < numCourses; i++) if (indegree[i] == 0) q.add(i);

int count = 0;
while (!q.isEmpty()) {
    int curr = q.poll();
    count++;
    for (int neighbor : adj.get(curr)) {
        if (--indegree[neighbor] == 0) q.add(neighbor);
    }
}
return count == numCourses;
```

Topic 6: Dynamic Programming & Backtracking

- **Backtracking Mental Model:** choose → explore → undo. Subsets (#78) → Combinations (#77) → Combination Sum (#39) → Permutations (#46) → Word Search (#79) → N-Queens (#51).
- **1D & 2D DP Framework:** State → Choice → Transition → Base Case.
- **Take/Skip (0/1 Knapsack):** #198 House Robber, #416 Partition Equal Subset Sum.
- **Unbounded Knapsack:** #322 Coin Change (min coins), #518 Coin Change II (ways).
- **2D Sequence DP:** #1143 Longest Common Subsequence, #72 Edit Distance.
- **Interval DP:** #312 Burst Balloons.

3. 60-90 Day Timetable & 7th Sem Exam Protocol

Phase	Dates	Topic Target	Daily Goal
Phase 1	Sept 22 - Oct 12	Arrays, Strings, Two Pointers, Monotonic Stack	3 problems/day + update streak app
Phase 2	Oct 13 - Nov 02	Linked Lists, Binary Trees, BST, Heaps	2-3 problems/day + notes
Phase 3	Nov 03 - Nov 23	Graphs, DSU, Dijkstra, 1D/2D DP, Backtracking	2 problems/day (Focus on pattern derivation)
Exam Gap	2-3 Weeks (End Sem)	Streak Maintenance & Revision	1 Easy problem/day (15 mins)
Phase 4	Nov 24 - Dec 15	Core 75 Timed Re-run & Off-Campus Applying	Timed mocks (20m/medium) + 3 Cold emails